28. Reverse Order of Items in Java — Quick Notes

Problem Statement

Given a list of integers, reverse the order of elements.

Example

Input : 10 20 30
Output: 30 20 10
Input : 3 8 10 12 15
Output: 15 12 10 8 3

---

1. Using Collections.reverse()

Java provides an inbuilt method.
Collections.reverse(list);

Example

List<Integer> list = Arrays.asList(10,20,30);
Collections.reverse(list);
Output: 30 20 10

Complexity

Time	Space
Θ(n)	O(1)

---

2. Using Swapping (Random Access Lists)

Applicable for:

ArrayList
Arrays

Idea

Swap:

First ↔ Last
Second ↔ Second Last
...
Until the middle is reached.

Example

10 20 30 40

Swap:

10 ↔ 40
20 ↔ 30

Result: 40 30 20 10

Algorithm

```
int left = 0;
int right = list.size()-1;

while(left < right){
   swap(left,right);

   left++;
   right--;
}
```

Complexity

Time	Space
Θ(n)	O(1)

---

3. Using Stack

Idea

Stack follows: LIFO (Last In First Out)
Last inserted element comes out first.

Algorithm

Step 1

Push all list elements into stack.
List
10 20 30
↓
Stack

Top
↓
30
20
10

Step 2

Pop elements from stack.
Pop → 30
Pop → 20
Pop → 10
Store back into list.

Example

Original List: 10 20 30

Push to Stack:
Top

↓
30
20
10
Pop back: 30 20 10

Code

```
Deque<Integer> stack =
      new ArrayDeque<>();

for(Integer x : list)
  stack.push(x);

for(int i=0;i<list.size();i++)
  list.set(i, stack.pop());
```

Complexity

Operation	Complexity
Push n items	Θ(n)
Pop n items	Θ(n)

Total: Θ(n)
Auxiliary Space: Θ(n)

Why Use Stack?

Because Stack naturally reverses order.
Insert:

10
20
30

Removal:

30
20
10
Hence: Last item becomes First item

Is Stack Efficient Here?

For ArrayList

❌ Not ideal.
Because ArrayList supports:Random Access

Swapping is better.

Better Approach for ArrayList

Θ(n) Time

Θ(1) Space

using two pointers.

When is Stack Useful?

When Random Access is NOT available.

Examples:

- Singly Linked List
- Queue
- Iterator
- TreeSet traversal
- Any forward-only container

Iterator Example

Iterator provides only: Forward Traversal
Suppose: 10 20 30
Iterator can access: 10 → 20 → 30
Cannot directly access: 30 → 20 → 10

Solution:

Iterator
↓
Stack
↓
Reverse Order

TreeSet → Queue in Reverse Order

Suppose: TreeSet
10 20 30

Need: Queue
30 20 10

Algorithm:

TreeSet
↓
Push into Stack
↓
Pop from Stack
↓
Insert into Queue

Complexity

Time	Space
Θ(n)	Θ(n)

Stack Implementation Recommendation

Legacy Approach

Stack<Integer> stack = new Stack<>();
Problems:

- Legacy class
- Extends Vector
- Thread-safe
- Synchronization overhead

Recommended Approach

Deque<Integer> stack = new ArrayDeque<>();
Advantages:

- Faster
- No synchronization overhead
- Modern implementation

Deque Stack Operations

Stack Operation	Deque Method
Push	`push(x)`
Pop	`pop()`
Peek	`peek()`

Complexity of Stack Operations

Operation	Complexity
push()	O(1)*
pop()	O(1)*
peek()	O(1)

* Amortized O(1)

Which Reversal Method to Use?

Scenario	Recommended Approach
ArrayList	Swapping
Arrays	Swapping

Iterator	Stack
Singly Linked List	Stack
Queue	Stack
TreeSet	Stack
Quick Built-in Solution	`Collections.reverse()`

Important Interview Points

- `Collections.reverse()` reverses a list directly.
- Stack reversal uses **LIFO property**.
- Stack reversal requires **Θ(n) auxiliary space**.
- Swapping is preferred when **random access** is available.
- Stack is useful for **forward-only traversal structures**.
- Prefer:

Deque<Integer> stack = new ArrayDeque<>();

instead of: Stack<Integer> stack = new Stack<>();

Quick Revision

Concept	Remember
Built-in Reverse	`Collections.reverse()`
Random Access Available	Use Swapping
Forward Traversal Only	Use Stack
Stack Property	LIFO
Recommended Stack	`ArrayDeque`
Stack Reversal Time	Θ(n)
Stack Reversal Space	Θ(n)
Swapping Space	Θ(1)